EXAM HEIST

EXAM HEIST

Premium exam material

Get certification quickly with the Exam Heist Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates

First attempt guaranteed success.

https://www.ExamHeist.com

Oracle

(1z0-829)

Java SE 17 Developer

Total: **84 Questions**

Link: https://examheist.com/papers/oracle/1z0-829

Question: 1 **Exam Heist**

Given the code fragment:

```
class Test {
    volatile int x = 1;
    AtomicInteger xObj = new AtomicInteger(1);
}
```

and

```
public static void main(String[] args) {
    Test t = new Test();
    Runnable r1 = () -> {
        Thread trd = Thread.currentThread();
        while (t.x < 3 ) {
            System.out.print(trd.getName()+" : "+t.x+" : ");
            t.x++;
        }
    };
    Runnable r2 = () -> {
        Thread trd = Thread.currentThread();
        while (t.xObj.get() < 3) {
            System.out.print(trd.getName()+" : "+t.xObj.get()+" : ");
            t.xObj.getAndIncrement();
        }
    };
    Thread t1 = new Thread(r1,"t1");
    Thread t2 = new Thread(r2,"t2");
    t1.start();
    t2.start();
}
```

Which is true?

A.The program prints t1 : 1 : t2 : 1: t1 : 2 : t2 : 2 : in random order.

B.The program prints t1 : 1 : t2 : 1: t1 : 2 : t2 : 2 :

C.The program prints t1 : 1 : t2 : 1: t1 : 1 : t2 : 1 : indefinitely.

D.The program prints an exception.

Answer: A

Explanation:

Answer: A. The program prints t1 : 1 : t1 : 2 : t2 : 1 : t2 : 2 : in random order.

The program launches two threads:

1. **t1**: Uses t.x (a volatile int). It prints t.x when it's 1 and 2, then increments it. The volatile keyword ensures visibility of changes to x across threads.
2. **t2**: Uses t.xObj (an AtomicInteger). It prints t.xObj when it's 1 and 2, then atomically increments it. AtomicInteger guarantees thread-safe increments.

Both threads will execute their loops twice (for values 1 and 2) and then terminate. Since t1 and t2 run concurrently, the exact order in which their System.out.print statements interleave is **non-deterministic**. Each thread will produce two lines of output, but the sequence of these four lines will vary with each execution.

Therefore, the program will print "t1 : t.x+ : 1 : ", "t1 : t.x+ : 2 : ", "t2 : t.xObj.get()+ : 1 : ", and "t2 : t.xObj.get()+ : 2 : " but in a random order.

Answer: A. The program prints t1 : 1 : t1 : 2 : t2 : 1 : t2 : 2 : in random order.

Question: 2 **Exam Heist**

Which statement is true?

A.IllegalStateException is thrown if a thread in waiting state is moved back to runnable.

B.A thread in waiting state consumes CPU cycles.

C.After the timed wait expires, the waited thread moves to the terminated state.

D.A thread in waiting state must handle InterruptedException.

Answer: D

Explanation:

D. A thread in waiting state must handle InterruptedException.

If a thread is in a waiting state due to methods like wait(), sleep(), or join(), it can be interrupted, which causes an InterruptedException to be thrown. This must be either caught or declared in the method signature.

Question: 3 **Exam Heist**

Daylight Saving Time (DST) is the practice of advancing clocks at the start of spring by one hour and adjusting them backward by one hour in autumn.
Considering that in 2021, DST in Chicago (Illinois) ended on November 7th at 2 AM, and given the fragment:

```
ZoneId zoneID = ZoneId.of("America/Chicago");
ZonedDateTime zdt = ZonedDateTime.of(
    LocalDate.of(2021, 11, 7),
    LocalTime.of(1, 30),
    zoneID
);
ZonedDateTime anHourLater = zdt.plusHours(1);
System.out.println(zdt.getHour() == anHourLater.getHour());
System.out.print(zdt.getOffset().equals(anHourLater.getOffset()));
```

What is the output?

A.true
true

B.false
true

C.true
false

D.false
false

Answer: C

Question: 4 **Exam Heist**

Given the code fragment:

```
// Login time:2021-01-12T21:58:18.817Z
Instant loginTime = Instant.now();
Thread.sleep(1000);

// Logout time:2021-01-12T21:58:19.880Z
Instant logoutTime = Instant.now();

loginTime = loginTime.truncatedTo(ChronoUnit.MINUTES);   // line n1
logoutTime = logoutTime.truncatedTo(ChronoUnit.MINUTES);

if (logoutTime.isAfter(loginTime))
    System.out.println("Logged out at: " + logoutTime);
else
    System.out.println("Can't logout");
```

What is the result?

A.Can't logout

B.Logged out at: 2021-01-12T21:58:00z

C.A compilation error occurs at line n1.

D.Logged out at: 2021-01-12T21:58:19.880z

Answer: A

Question: 5 **Exam Heist**

Given the code fragment:

```
Duration duration = Duration.ofMillis(5000);
System.out.print(duration);
duration = Duration.ofSeconds(60);
System.out.print(duration);
Period period = Period.ofDays(6);
System.out.print(period);
```

What is the result?

A.PT5000SPT60MP6D

B.PT5SPT1MP6D

C.5S1M6D

D.5000S60M6D

Answer: B

Question: 6 **Exam Heist**

Given the code fragment:

```
record Product(int pNumber, String pName) {
    int regNo = 100;
    public int getRegNumber() {
        return regNo;
    }
}

public class App {
    public static void main(String[] args) {
        Product p1 = new Product (1111, "Ink Bottle");
    }
}
```

Which action enables the code to compile?

A.Replace record with void.

B.Replace record with class.

C.Remove the regNo initialization statement.

D.Make the regNo variable public.

E.Make the regNo variable static.

Answer: E

Question: 7 **Exam Heist**

Given:

```
public class Weather {
    public enum Forecast {
        SUNNY, CLOUDY, RAINY;
        @Override
        public String toString() { return "SNOWY";}
    }

    public static void main(String[] args) {
        System.out.print(Forecast.SUNNY.ordinal() + " ");
        System.out.print(Forecast.valueOf("cloudy".toUpperCase()));
    }
}
```

What is the result?

A.0 CLOUDY

B.1 SNOWY

C.0 SNOWY

D.1 RAINY

E.Compilation fails

Answer: C

Question: 8 **Exam Heist**

Given the code fragment:

```
List<String> specialDays = List.of("NewYear","Valentines","Spring","Labour");
System.out.print(specialDays.stream().allMatch(s ->s.equals("Labour")));
System.out.print(" " + specialDays.stream().anyMatch(s ->s.equals("Labour")));
System.out.print(" " + specialDays.stream().noneMatch(s -> s.equals("Halloween")));
System.out.print(" " +specialDays.stream().findFirst());
```

What is the result?

A.0 1 1 0

B.false true true Optional[NewYear]

C.0 1 0 Optional[NewYear]

D.true true false NewYear

Answer: B

Question: 9 **Exam Heist**

Given:

```
public class App{
    String name;
    public App(String name){
        this.name = name;
    }
    public static void main(String args[]) {
        App t1= new App("t1");
        App t2= new App("t2");
        t1 = t2;
        t1 = null;
        System.out.println("GC");
    }
}
```

Which statement is true while the program prints GC?

A.Both the objects previously referenced by t1 are eligible for garbage collection.

B.None of the objects are eligible for garbage collection.

C.Only the object referenced by t2 is eligible for garbage collection.

D.Only one of the objects previously referenced by t1 is eligible for garbage collection.

Answer: D

Question: 10 **Exam Heist**

Given:

```
public class Main {
    void print(int i){
        System.out.println("hello");
    }
    void print(long j){
        System.out.println("there");
    }

    public static void main(String[] args) {
        new Main().print(0b1101_1010);
    }
}
```

What is the result?

A.Compilation fails.

B.there

C.hello

D.A NumberFormatException is thrown.

Answer: C

Question: 11 **Exam Heist**

Given the code fragment:

```
abstract sealed interface SInt permits Story, Art {
default String getTitle() { return "Book Title" ; }
}
```

Which set of class definitions compiles?

A.interface Story extends SInt
interface Art extends SInt

B.public interface Story extends SInt
public interface Art extends SInt

C.sealed interface Story extends SInt
non-sealed class Art implements SInt

D.non-sealed interface Story extends SInt
non-sealed interface Art extends SInt

E.non-sealed interface Story extends SInt
class Art implements SInt

Answer: D

Question: 12 **Exam Heist**

Given:

```
class A {public void mA() {System.out.println("mA");}}
class B extends A {public void mB() {System.out.println("mB");}}
class C extends B {public void mC() {System.out.println("mC");}}

public class App {
    public static void main(String[] args) {
        A bobj = new B();
        A cobj = new C();
        if (cobj instanceof B v) {
            v.mB();
            if (v instanceof C v1) { v1.mC(); }
        } else {
            cobj.mA();
        }
    }
}
```

What is the result?

A.mA

B.mB
mA

C.mB

D.mB
mC

Answer: D

Question: 13 **Exam Heist**

Given the code fragment:

```
Stream<String> s1 = Stream.of("A", "B", "C", "B");
Stream<String> s2 = Stream.of("A", "D", "E");
Stream.concat(s1, s2).parallel().distinct().forEach(element -> System.out.print(element));
```

What is the result?

A.ABBCDE // the order of elements is unpredictable

B.ABCDE

C.ABCDE // the order of elements is unpredictable

D.ADEABCB // the order of elements is unpredictable

Answer: C

Question: 14 **Exam Heist**

Given:

```
public class Test {
   public static void main(String[] args) {
       List<String>  elements =
             Arrays.asList("car", "truck", "car",
                           "bicycle", "car", "truck", "motorcycle");
       Map<String, Long>  outcome =
             elements.stream().collect(Collectors.groupingBy(Function.identity(), Collectors.counting() ) );
       System.out.println(outcome);
   }
}
```

What is the result?

A. bicycle=7, car=7, motorcycle=7, truck=7

B. 3:bicycle, 0:car, 6:motorcycle, 5:truck

C.Compilation fails.

D. bicycle=1, car=3, motorcycle=1, truck=2

E. bicycle, car, motorcycle, truck

Answer: D

Question: 15 **Exam Heist**

Which statement is true about modules?

A.Only unnamed modules are on the module path.

B.Automatic and named modules are on the module path.

C.Only automatic modules are on the module path.

D.Only named modules are on the module path.

E.Automatic and unnamed modules are on the module path.

Answer: B

Explanation:

The correct answer is B: Automatic and named modules are on the module path. Let's break down why.

Java's module system (introduced in Java 9) provides a way to organize and encapsulate code, improving security and maintainability. The module path is where the Java runtime looks for modules when resolving dependencies.

Named modules are modules defined with a module-info.java file. This file explicitly declares the module's dependencies and which packages it exports. These are clearly defined, well-behaved modules and always reside on the module path.

Automatic modules are JAR files that have been placed on the module path but do not contain a module-info.java file. The Java runtime automatically turns these JARs into modules based on the JAR's name. While they are "automatic," they still exist on the module path for the Java runtime to discover and use them.

Unnamed modules, on the other hand, are classes that reside on the classpath, not the module path. The classpath is still used in modular Java, primarily for legacy code and dependencies that haven't been modularized. Unnamed modules can access classes from named and automatic modules, but named and automatic modules generally should not depend directly on the unnamed module (it can be tricky to manage). The requires transitive is for other modules in your module path, it won't help the module access unnamed modules.

Therefore, only modules found on the module path are considered by the module system during dependency resolution. These include both named and automatic modules. Unnamed modules are outside the module system's strict dependency management scope.

In summary, the module path serves as the defined location for both the structured, explicitly declared named modules and the automatically converted automatic modules, enabling the module system to function correctly.

Further reading:

The State of the Module System
Understanding Java 9 Modules (Baeldung is generally a good resource).

Question: 16 **Exam Heist**

Given the code fragment:

```
// line n1
String input = console.readLine("Input a number: ");
int number = Integer.parseInt(input);

if (number % 2 == 0) {
    System.out.println(number + " is even.");
} else {
    System.out.println(number + " is odd");
}
```

Which code at line n1, obtains the java.io.Console object?

A.Console console = Console.getInstance();
B.Console console = System.console();
C.Console console = new Console(System.in);

D.Console console = new Console(new InputStreamReader(System.in));

E.Console console = System.console(System.in);

Answer: B

Question: 17 **Exam Heist**

Given the content of the in.txt file:
0123456789
and the code fragment:

```
char[] buffer = new char[8];
int count = 0;
try(FileReader in = new FileReader("in.txt");
    FileWriter out = new FileWriter("out.txt")) {
        while((count = in.read(buffer)) != -1) {
           out.write(buffer);
        }
}
```

What is the content of the out.txt file?

A.012345678

B.01234567801234

C.01234567

D.0123456789

E.012345678901234

F.0123456789234567

Answer: F

Question: 18 **Exam Heist**

Given the code fragment:

```
String myStr = "Hello Java 17";
String myTextBlk1 = """
                    Hello Java 17""";
String myTextBlk2 = """
                    Hello Java 17
                    """;
System.out.print(myStr.equals(myTextBlk1)+":");
System.out.print(myStr.equals(myTextBlk2)+":");
System.out.print(myTextBlk1.equals(myTextBlk2)+":");
System.out.println(myTextBlk1.intern() == myTextBlk2.intern());
```

What is the result?

A.true:true:false:false

B.true:false:true:false

C.true:false:true:true

D.true:false:false:false

Answer: D

Question: 19 **Exam Heist**

Given the code fragment:

```
String a = "Hello! Java";
System.out.print(a.indexOf("Java"));
a.replace("Hello!", "Welcome!");
System.out.print(a.indexOf("Java"));
StringBuilder b = new StringBuilder(a);
System.out.print(b.indexOf("Java"));
```

What is the result?

A.888

B.8109

C.7107

D.777

E.71010

F.81111

Answer: D

Question: 20 **Exam Heist**

Given:

```
public class Test {
   public static void main(String[] args) {
      final int x = 2;
      int y = x;
      while (y<3) {
         switch (y) {
            case 0+x:
            y++;
            case 1:
            y++;
         }
      }
      System.out.println(y);
   }
}
```

What is the result?

A.4

B.2

C.6

D.Compilation fails.

E.A runtime exception is thrown.

F.Nothing is printed because of an indefinite loop.

G.5

H.3

Answer: A

Question: 21 **Exam Heist**

Given the code fragment:

```
Integer rank = 4;
switch (rank) {
    case 1,4 -> System.out.println("Range1");
    case 5,8 -> System.out.println("Range2");
    case 9,10 -> System.out.println("Range3");
    default -> System.out.println("Not a valid rank.");
}
```

What is the result?

A.Range1 -

Range2 -

Range3 -
Range1 -
Not a valid rank.

B.Range1 -
Not a valid rank.

C.Range1 -

Range2 -
Range3

D.Range1

Answer: D

Question: 22 **Exam Heist**

Given the code fragment:
List<Integer> listOfNumbers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
Which code fragment returns different values?

A.int sum = listOfNumbers.stream().reduce(0, Integer::sum) + 5;

B.int sum = listOfNumbers.parallelStream().reduce(0, Integer::sum) + 5 ;

C.int sum = listOfNumbers.parallelStream().reduce((m, n) -> m + n).orElse(5) + 5;

D.int sum = listOfNumbers.parallelStream().reduce(5, Integer::sum);

E.int sum = listOfNumbers.stream().reduce(5, (a, b) -> a+ b);

Answer: D

Explanation:

Here's a detailed justification for why option D returns a different value than the other options, focusing on how reduce operates in both sequential and parallel streams, and emphasizing the implications of the identity parameter.

Options A, B, and C, while using different approaches to the reduce operation, all effectively calculate the sum of the elements in the listOfNumbers list (1 + 2 + 3 + ... + 10 = 55) and then add 5 to the result. Therefore, each of them results in 60. Options A and B both start with an initial value of 0 and cumulatively sum the elements of the stream. Option A does so in a sequential manner, while option B does it in parallel. Option C uses a different overload of reduce that doesn't require an identity element, instead returning an Optional<Integer>. If

the stream is empty, the orElse(5) provides the default result of 5, but with a non-empty stream, it results in the sum of all elements. The +5 adds 5 to the sum.

Option D, however, utilizes the reduce(identity, accumulator) overload in a parallel stream, which is where the difference arises. The identity element (5 in this case) is not just used as the initial value for the first accumulation. In a parallel stream, the identity is combined with each element across multiple threads before the partial results are combined. This causes the identity to be added multiple times. Specifically, in this case, the accumulator Integer::sum means each core will start with the identity 5 and add elements in parallel. Then, the partial results will be added again, implicitly adding the identities used by each core. Therefore the sum is not simply the sum of the list plus 5, but a much larger number.

Option E behaves like A. It performs the accumulation sequentially starting with the identity value of 5. It differs from option D due to its sequential execution.

In summary, options A, B, C and E correctly sum the numbers and add 5, whereas D adds 5 multiple times due to the parallel processing and the behavior of the identity element within the reduce operation. The multiple additions of the identity result in a different final value.

Oracle Java Documentation on Stream.reduce():
https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/stream/Stream.html#reduce(U,java.util.funct
Understanding Identity in Java Stream Reduce: https://www.baeldung.com/java-stream-reduce

Question: 23 **Exam Heist**

Given:

```
public class App {
    public int x = 100;

    public static void main(String[] args) {
        int x = 1000;
        App t = new App();
        t.myMethod(x);
        System.out.println(x);
    }
    public void myMethod(int x) {
        x++;
        System.out.println(x);
        System.out.println(this.x);
    }
}
```

What is the result?

A.100
100
1000

B.101
101
1000

C.1001
100
1000

D.1001
1001
1000

Answer: C

Question: 24 **Exam Heist**

Given the code fragment:

```
List lst = new ArrayList();
lst.add("e1");
lst.add("e3");
lst.add("e2");

int x1 = Collections.binarySearch(lst, "e3");
System.out.println(x1);
Collections.sort(lst);
int x2 = Collections.binarySearch(lst, "e3");
System.out.println(x2);

Collections.reverse(lst);
int x3 = Collections.binarySearch(lst, "e3");
System.out.println(x3);
```

What is the result?

A.1
2
-4

B.2
-2

C.1
1
1

D.2
2
0

Answer: A

Question: 25 **Exam Heist**

Given the code fragment:

```
String s = "10_00";
Integer s2 = 10_00;
// Line n1
System.out.println(res);
```

Which two statements at Line n1 independently enable you to print 1250? (Choose two.)

A.Integer res = 250 + Integer.parseInt(s);

B.Integer res = 250 + Integer.valueOf(s);

C.Integer res = 250 + s2;

D.Integer res = 250;
res =+ s2;

E.Integer res = 250 + s;

F.Integer res = 250 + Integer(s2);

Answer: CD

Question: 26 **Exam Heist**

Given the code fragment:

```
int a = 2;
int b = ~a;
int c = a^b;
boolean d = a < b & a > c++;
System.out.println(d + " " + c);
boolean e = a > b && a > c++;
System.out.println(e + " " + c);
```

What is the result?

A.false 0
true 1

B.false 1
false 2

C.true 1
false 2

D.false 1
true 2

Answer: A

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

But Wait

I wanted to let you know that there is more content available in the full version.
The full paper contains additional sections and information that you may find helpful, and I encourage you to download it to get a more comprehensive and detailed view of all the subject matter.

Download Full Version Now

Future is Secured
100% Pass Guarantee

24/7 Customer Support
Mail us - support@examheist.com

Verified Updates
Lifetime Updates!

Total: **84 Questions**
Link: https://examheist.com/papers/oracle/1z0-829